

Algorithmique

Coût et Complexité des algorithmes

Cédric Lhoussaine

Université de Lille / IUT de Lille

R5.A.07



- **Organisation**

1. 4 cours (1h)
2. 4 TD (1h30)
3. 4 TP (1h30)
4. DS (2h)

- **Organisation**

1. 4 cours (1h)
2. 4 TD (1h30)
3. 4 TP (1h30)
4. DS (2h)

- **Programme**

Complexité algorithmique, tris, récursivité, algorithmes sur les graphes / arbres

- **Organisation**

1. 4 cours (1h)
2. 4 TD (1h30)
3. 4 TP (1h30)
4. DS (2h)

- **Programme**

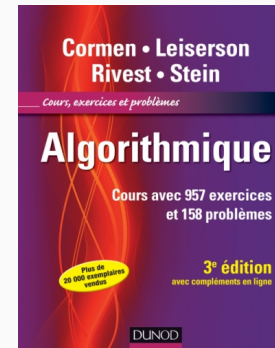
Complexité algorithmique, tris, récursivité, algorithmes sur les graphes / arbres

- **Bibliographie**

Algorithmique

Thomas H. Cormen, Charles Leiserson,
Ronald Rivest, Clifford Stein

Editions Dunod



Fournitures pour chaque CM, TD et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Méthodologie de travail: recommandations

Fournitures pour chaque CM, TD et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → **facilitent la mémorisation** et économisent le travail à la maison!

Méthodologie de travail: recommandations

Fournitures pour chaque CM, TD et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → **facilitent la mémorisation** et économisent le travail à la maison!

Pour chaque CM, TD, TP:

- revoir le(s) cours précédents
- finir les TD/TPs

Méthodologie de travail: recommandations

Fournitures pour chaque CM, TD et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → **facilitent la mémorisation** et économisent le travail à la maison!

Pour chaque CM, TD, TP:

- revoir le(s) cours précédents
- finir les TD/TPs

Tenir à distance tout objet de distraction massive (smartphone, voisin bavard, etc.)

- Introduction et rappel
- Coût d'un algorithme
- Complexité algorithmique
- Complexités usuelles et ordres de grandeur

Introduction et rappels

Définition Un *algorithme* est une séquence d'étapes de calculs qui résout un problème, c'est-à-dire qui transforme une entrée en une sortie.

Définition Un *algorithme* est une séquence d'étapes de calculs qui résout un problème, c'est-à-dire qui transforme une *entrée* en une *sortie*.

Exemple (Problème de tri) Qu'est-ce qu'une *entrée*, une *sortie* ?

- *Entrée*: une suite de n nombres: $\langle a_1, \dots, a_n \rangle$
- *Sortie*: une permutation $\langle a'_1, \dots, a'_n \rangle$ des nombres d'entrée telle que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Définition Une *instance de problème* est un exemple particulier d'entrée pour ce problème.

Définition Une *instance de problème* est un exemple particulier d'entrée pour ce problème.

Exemple (Instances du problème de tri) Deux instances du problème de tri:

- $\langle 10, 15, 3 \rangle$ (dont la sortie sera $\langle 3, 10, 15 \rangle$)
- $\langle 2, 123, 4, 10 \rangle$ (dont la sortie sera $\langle 2, 4, 10, 123 \rangle$)

Deux propriétés importantes d'algorithmes

Un algorithme doit être **correct** et **efficace**!

Deux propriétés importantes d'algorithmes

Définition Un algorithme est *correct* si, pour chaque entrée, il se termine en produisant la bonne sortie.

Deux propriétés importantes d'algorithmes

Définition Un algorithme est *correct* si, pour chaque entrée, il se termine en produisant la bonne sortie.

⚠ Il peut parfois être utile d'avoir des algorithmes incorrects (en particulier, si le taux d'erreurs peut être contrôlé)

Deux propriétés importantes d'algorithmes

Définition Un algorithme est *correct* si, pour chaque entrée, il se termine en produisant la bonne sortie.

⚠ Il peut parfois être utile d'avoir des algorithmes incorrects (en particulier, si le taux d'erreurs peut être contrôlé)

cf. cours suivant...

Deux propriétés importantes d'algorithmes

Définition L'*efficacité* d'un algorithme est sa capacité à résoudre un problème en minimisant son temps d'exécution (et/ou son utilisation de l'espace mémoire).

(on ne traitera pas l'usage de l'espace mémoire dans ce cours)

Coût d'un algorithme

Deux propriétés importantes d'algorithmes

Comment mesurer le temps d'exécution (ou coût) d'un algorithme ? (Wooclap)

Exemple: coût de la somme des éléments d'un tableau

Algorithm 1. – Somme des éléments d'un tableau d'entiers

```
1: function SOMME-TABLEAU(tab)
2:   somme  $\leftarrow$  0
3:   for  $i \leftarrow 0$  to longueur(tab) - 1 do
4:     somme  $\leftarrow$  somme + tab[i]
5:   end
6:   return somme
7: end
```

Calcul au tableau...

Le temps d'exécution dépend:

- du temps d'exécution de chaque instruction *élémentaire*
- de la taille de l'entrée (= nombre d'entiers, de chaînes de caractères, etc.)

Le temps d'exécution dépend:

- du temps d'exécution de chaque instruction *élémentaire*
- de la taille de l'entrée (= nombre d'entiers, de chaînes de caractères, etc.)

On appelle **instruction élémentaire**, une instruction dont le coût ne dépend que des caractéristiques physiques de la machine, du langage, de l'OS, etc. et pas de l'entrée.

Mesure du coût d'un algorithme

Le temps d'exécution dépend:

- du temps d'exécution de chaque instruction *élémentaire*
- de la taille de l'entrée (= nombre d'entiers, de chaînes de caractères, etc.)

On appelle **instruction élémentaire**, une instruction dont le coût ne dépend que des caractéristiques physiques de la machine, du langage, de l'OS, etc. et pas de l'entrée.

Remarque Le **temps d'exécution** (ou **coût**) d'un algorithme est une fonction, généralement notée $T(n)$, qui dépend de la taille n de l'entrée et de constantes.

Y a-t-il d'autres facteurs dont peut dépendre l'efficacité d'un algorithme ? Autrement dit:

Y a-t-il d'autres facteurs dont peut dépendre l'efficacité d'un algorithme ? Autrement dit:

Pour deux entrées de même taille et les mêmes conditions logicielles et matérielles d'exécution, un algorithme aura-t-il le même temps d'exécution pour ces deux entrées ? (Wooclap)

Exemple: coût de la recherche linéaire dans un tableau

Algorithm 2. – Recherche Linéaire

```
1: function RECHERCHE-LINÉAIRE(tab, val)
2:   trouve  $\leftarrow$  false
3:   for  $i \leftarrow 0$  to longueur(tab) - 1 do
4:     if tab[ $i$ ] = val then
5:       |   trouve  $\leftarrow$  true
6:     end
7:   end
8:   if trouve then
9:     |   return  $i$ 
10:  else
11:    |   return -1
12:  end
13: end
```

Calcul au tableau...

- $T^{\max}(n)$: **coût dans le pire des cas** (ou moins favorable), i.e. coût maximal pour toute entrée de taille n . C'est un *majorant* du coût de l'algorithme.

Mesure du coût d'un algorithme

- $T^{\max}(n)$: **coût dans le pire des cas** (ou moins favorable), i.e. coût maximal pour toute entrée de taille n . C'est un *majorant* du coût de l'algorithme.
- $T^{\min}(n)$: **coût dans le meilleur des cas** (ou plus favorable). Un algorithme lent sur des données de taille n , peut exceptionnellement être très rapide (ie. pour certaines données de taille n). C'est un *minorant* de l'algorithme.

Mesure du coût d'un algorithme

- $T^{\max}(n)$: **coût dans le pire des cas** (ou moins favorable), i.e. coût maximal pour toute entrée de taille n . C'est un *majorant* du coût de l'algorithme.
- $T^{\min}(n)$: **coût dans le meilleur des cas** (ou plus favorable). Un algorithme lent sur des données de taille n , peut exceptionnellement être très rapide (ie. pour certaines données de taille n). C'est un *minorant* de l'algorithme.
- $T^{\text{moy}}(n)$: **coût en moyenne**, i.e. temps moyen sur toutes les données de taille n (nécessite en général une hypothèse sur la distribution statistique des entrées).

Complexité Algorithmique (ou asymptotique)

Inconvénient: le coût d'un algorithme est donné en fonction de constantes qui dépendent de l'environnement d'exécution dont on aimerait s'affranchir.

Question: ces constantes sont-elles vraiment significantes pour des entrées de grande taille ?

Supposons deux algorithmes:

- A_1 de coût $T_1(n) = c * n$
- A_2 de coût $T_2(n) = c * n^2$

où c est une constante qui dépend de l'environnement d'exécution E .

Supposons deux algorithmes:

- A_1 de coût $T_1(n) = c * n$
- A_2 de coût $T_2(n) = c * n^2$

où c est une constante qui dépend de l'environnement d'exécution E .

Quel est l'algorithme le plus rapide ? (Wooclap)

On suppose $c = 1$ dans l'environnement E .

On suppose $c = 1$ dans l'environnement E .

Améliorons A_2 en l'exécutant dans un environnement E' 2 fois plus efficace que E .

Donc, dans E' , $c =$

On suppose $c = 1$ dans l'environnement E .

Améliorons A_2 en l'exécutant dans un environnement E' 2 fois plus efficace que E .

Donc, dans E' , $c = \frac{1}{2}$.

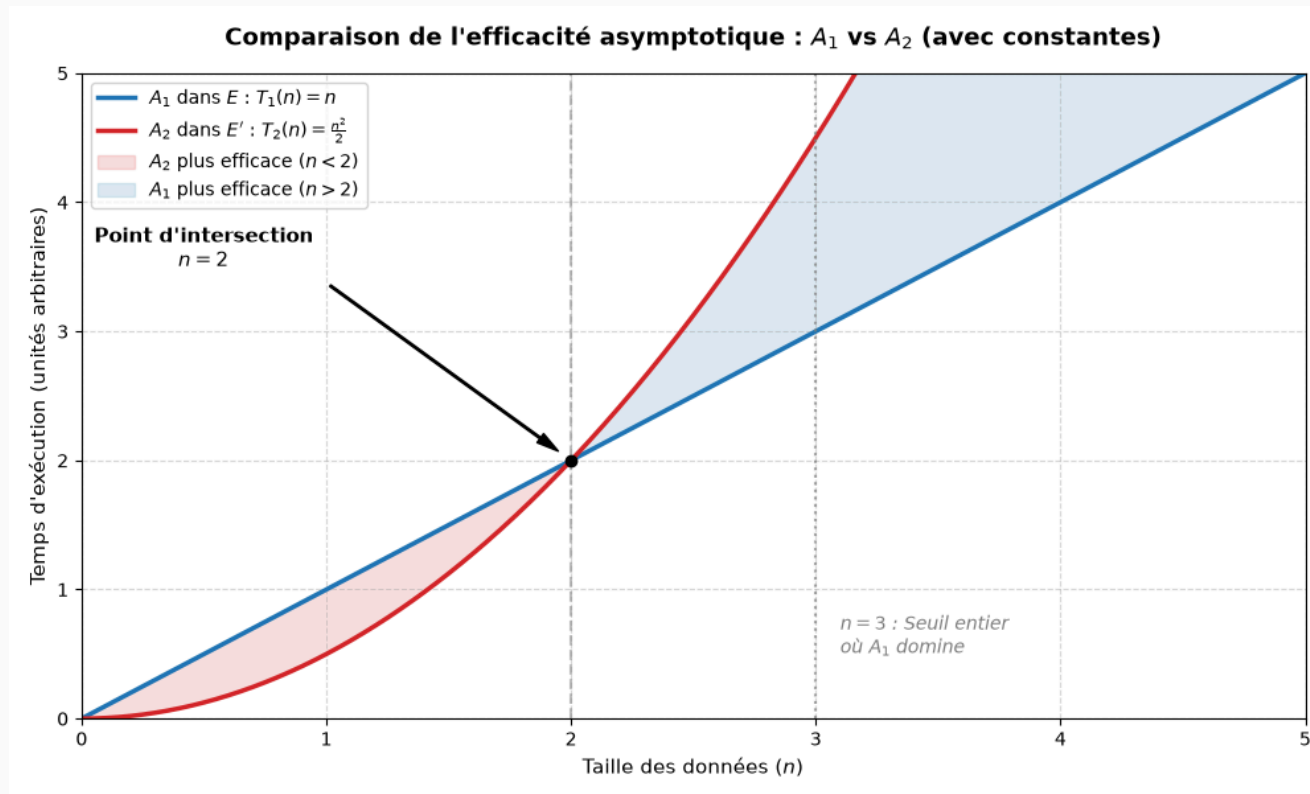
On suppose $c = 1$ dans l'environnement E .

Améliorons A_2 en l'exécutant dans un environnement E' 2 fois plus efficace que E .

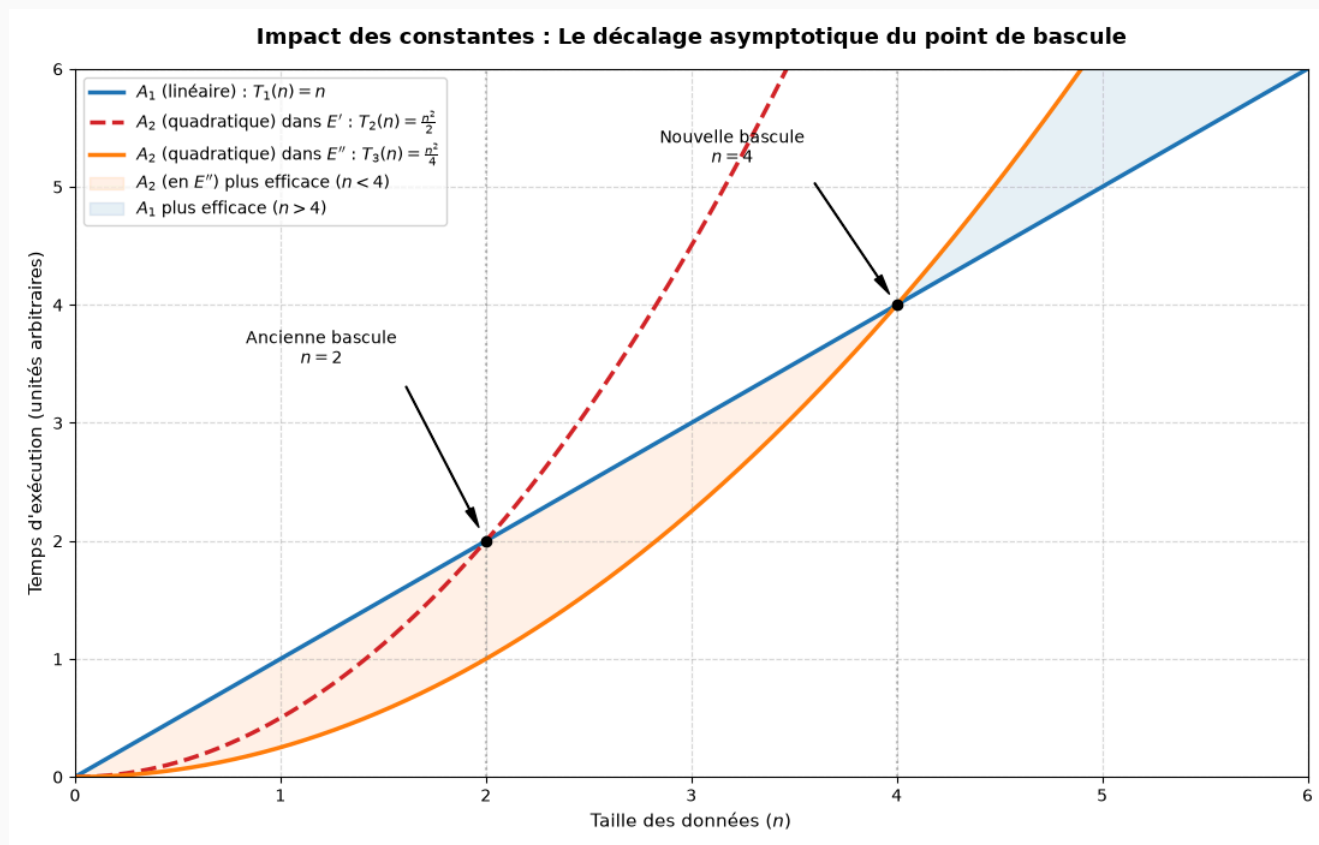
Donc, dans E' , $c = \frac{1}{2}$.

A_2 dans E' est-il plus efficace que A_1 dans E ? (Wooclap)

Coût et constantes



Doublons encore l'efficacité de l'environnement de $A_2 \Rightarrow T_2(n) = \frac{1}{4}n^2$



Remarque Pour de très grandes tailles d'entrée, A_1 sera toujours plus efficace que A_2 quelque soit son environnement d'exécution.

¹c'est-à-dire quand la taille de l'entrée devient très grande

Remarque Pour de très grandes tailles d'entrée, A_1 sera toujours plus efficace que A_2 quelque soit son environnement d'exécution.

Ainsi, les constantes sont *asymptotiquement*¹ insignifiantes.

¹c'est-à-dire quand la taille de l'entrée devient très grande

Remarque Pour de très grandes tailles d'entrée, A_1 sera toujours plus efficace que A_2 quelque soit son environnement d'exécution.

Ainsi, les constantes sont *asymptotiquement*¹ insignifiantes.

Mais comment se débarrasser de ces constantes ?...

¹c'est-à-dire quand la taille de l'entrée devient très grande

Un coût T_1 est *asymptotiquement meilleur* qu'un coût T_2 si, à partir d'une certaine taille d'entrée (n), T_1 est toujours inférieur à T_2 .

$T_1 = c_1 n$ est donc asymptotiquement meilleur que $T_2 = c_2 n^2$ (quelque soient c_1 et c_2 , et aussi petit que puisse être c_2)

Un coût T_1 est *asymptotiquement meilleur* qu'un coût T_2 si, à partir d'une certaine taille d'entrée (n), T_1 est toujours inférieur à T_2 .

$T_1 = c_1 n$ est donc asymptotiquement meilleur que $T_2 = c_2 n^2$ (quelque soient c_1 et c_2 , et aussi petit que puisse être c_2)

On désigne par $O(n^2)$, la classe des coûts asymptotiquement meilleurs que cn^2 (pour un c quelqueconque).

Un coût T_1 est *asymptotiquement meilleur* qu'un coût T_2 si, à partir d'une certaine taille d'entrée (n), T_1 est toujours inférieur à T_2 .

$T_1 = c_1 n$ est donc asymptotiquement meilleur que $T_2 = c_2 n^2$ (quelque soient c_1 et c_2 , et aussi petit que puisse être c_2)

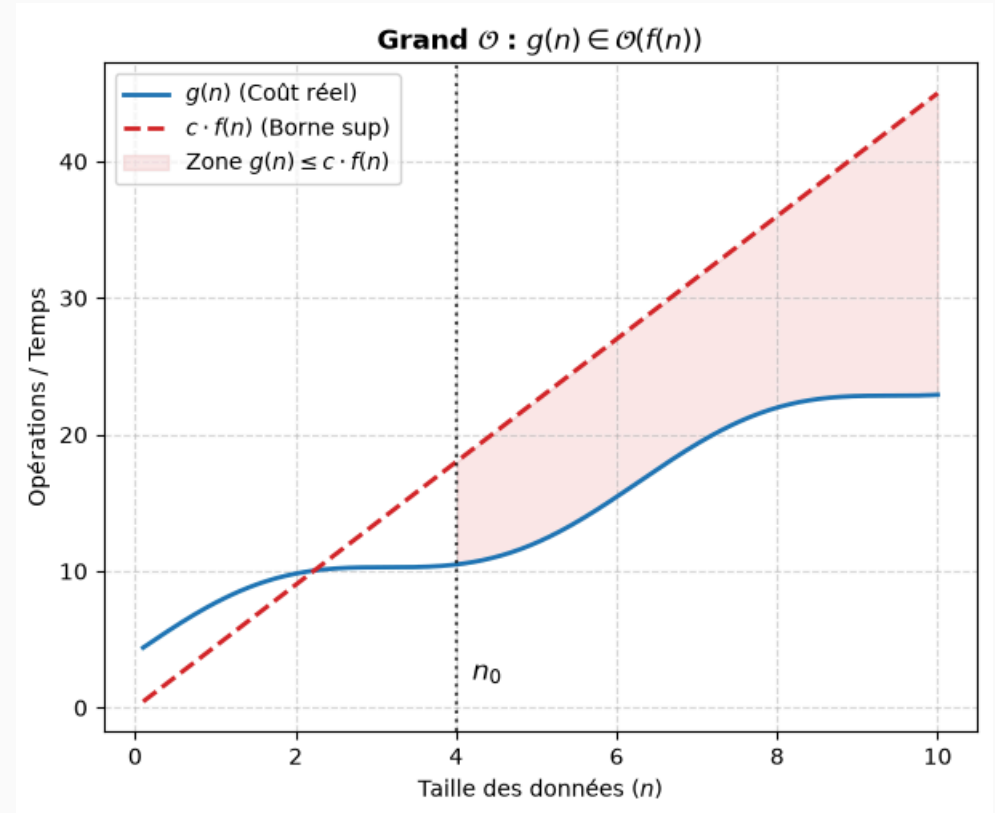
On désigne par $O(n^2)$, la classe des coûts asymptotiquement meilleurs que cn^2 (pour un c quelqueconque).

Ainsi, $n \in O(n^2)$, de même que $100n + 14 \in O(n^2)$ et $37 \in O(n^2)$.

Notation de Landau (O)

Définition Soit une fonction $f : \mathbb{N} \rightarrow \mathbb{R}^+$, on dénote $O(f)$ l'ensemble des fonctions g inférieure à f à partir d'un certain rang (à une constante multiplicative près):

$$O(f) = \{g \mid \exists c, n_0 \text{ t.q. } 0 \leq f(n) \leq cg(n) \text{ pour tout } n \geq n_0\}$$



Remarque Soit $g \in O(f)$,

- on écrira, par abus de notation, $g = O(f)$,
- et on dira que « g est en $O(f)$ »
- intuitivement, cela signifie que g finit toujours par être majorée par f

Notation de Landau (Ω)

De même, $\Omega(f)$ désigne les fonctions minorées par f .¹

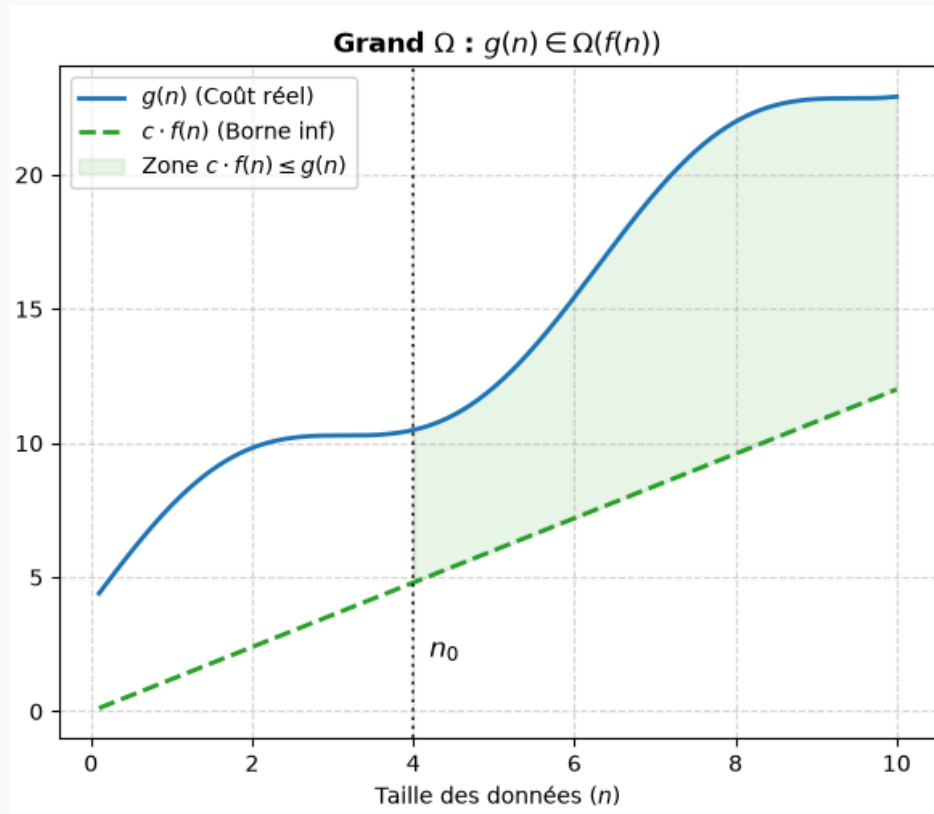
¹à une constante multiplicative près et à partir d'un certain rang.

Notation de Landau (Ω)

De même, $\Omega(f)$ désigne les fonctions minorées par f .¹

Définition Soit une fonction $f : \mathbb{N} \rightarrow \mathbb{R}^+$, on dénote $\Omega(f)$ l'ensemble des fonctions g supérieures à f à une constante multiplicative près et à partir d'un certain rang:

$$\Omega(f) = \{g \mid \exists c, n_0 \text{ t.q. } cg(n) \leq f(n) \text{ pour tout } n \geq n_0\}$$



¹à une constante multiplicative près et à partir d'un certain rang.

Notation de Landau (Θ)

et, $\Theta(f)$ désigne les fonctions encadrées par f .¹

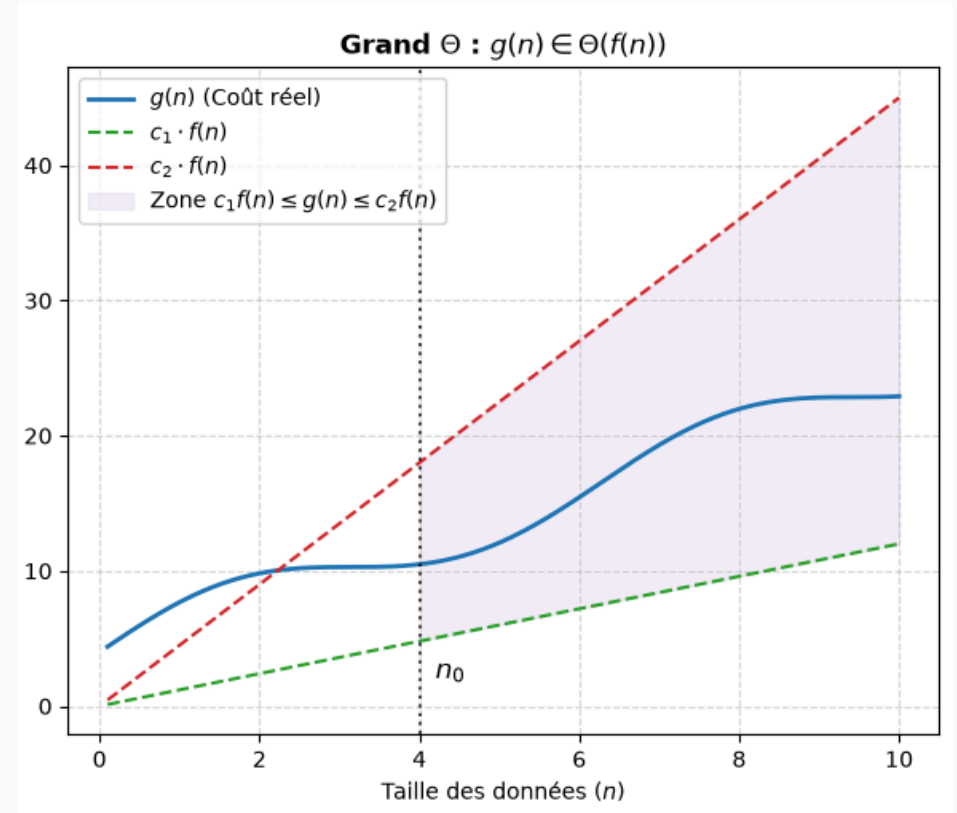
¹à des constantes multiplicatives près et à partir d'un certain rang.

Notation de Landau (Θ)

et, $\Theta(f)$ désigne les fonctions encadrées par f .¹

Définition Soit une fonction $f : \mathbb{N} \rightarrow \mathbb{R}^+$, on dénote $\Theta(f)$ l'ensemble des fonctions g encadrées par f à une constante multiplicative près et à partir d'un certain rang:

$$\Theta(f) = \{g \mid \exists c_1, c_2, n_0 \text{ t.q. } c_1 f(n) \leq g(n) \leq c_2 f(n) \text{ pour tout } n \geq n_0\}$$



¹à des constantes multiplicatives près et à partir d'un certain rang.

En général, on utilise la notation $O(f)$ pour estimer le coût d'un algorithme dans le pire cas $T^{\max}$ car elle permet de le majorer: pour toute entrée de grande taille, le coût ne dépassera pas $cf(n)$ (pour une certaine constante c).

Utilisation de la complexité asymptotique

En général, on utilise la notation $O(f)$ pour estimer le coût d'un algorithme dans le pire cas $T^{\max}$ car elle permet de le majorer: pour toute entrée de grande taille, le coût ne dépassera pas $cf(n)$ (pour une certaine constante c).

En général, on utilise la notation $\Omega(f)$ pour estimer le coût d'un algorithme dans le meilleur cas $T^{\min}$ car elle permet de le minorer: pour toute entrée de grande taille, le coût ne peut être inférieur à $cf(n)$ (pour une certaine constante c).

Utilisation de la complexité asymptotique

En général, on utilise la notation $O(f)$ pour estimer le coût d'un algorithme dans le pire cas $T^{\max}$ car elle permet de le majorer: pour toute entrée de grande taille, le coût ne dépassera pas $cf(n)$ (pour une certaine constante c).

En général, on utilise la notation $\Omega(f)$ pour estimer le coût d'un algorithme dans le meilleur cas $T^{\min}$ car elle permet de le minorer: pour toute entrée de grande taille, le coût ne peut être inférieur à $cf(n)$ (pour une certaine constante c).

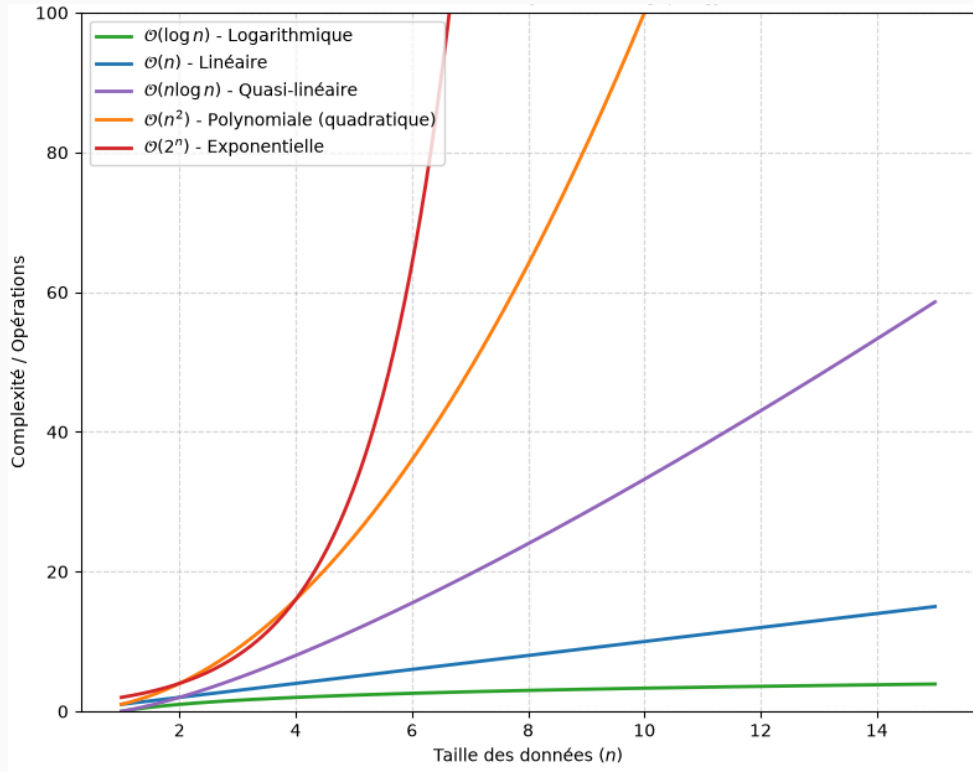
La notation $\Theta(f)$ donne l'estimation la plus précise d'un coût (dans le pire comme dans le meilleur cas).

Complexités usuelles et ordres de grandeur

Classer par vitesse de croissance les fonctions suivantes (Wooclap)

Complexités usuelles et ordres de grandeur

Ordre de grandeur des complexités :



$O(1)$	complexité constante
$O(\log(n))$	complexité logarithmique
$O(n)$	complexité linéaire
$O(n \log(n))$	complexité quasi-linéaire
$O(n^k)$	complexité polynomiale
$O(k^n)$	complexité exponentielle

Complexités usuelles et ordres de grandeur

Temps d'exécution pour différents coût algorithmique (en supposant une machine exécutant 10^9 opérations / seconde) :

	$\log n$	n	$n \log n$	n^2	n^3	2^n
10^2	7 ns	100 ns	$0.7 \mu s$	$10 \mu s$	1 ms	$4 \cdot 10^{13}$ années
10^3	10 ns	$1 \mu s$	$10 \mu s$	1 ms	1 s	10^{292} années
10^4	13 ns	$10 \mu s$	$133 \mu s$	100 ms	17 s	
10^5	17 ns	$100 \mu s$	2 ms	10 s	11.6 jours	
10^6	20 ns	1 ms	20 ms	17 mn	32 années	